

Slide-onderbouwing

Geautomatiseerde security assessment van MCP-servers

Verdediging bachelorproef — Arthur Van Oudenhove

Per slide: de tekst op de deck en de onderbouwing uit de bachelorproef (FINAL-versie: precisie-framing, één evaluatiecase). Bronverwijzingen tussen haakjes zijn de \autocite-keys uit bachproef.bib.

Slide 2. De poster

Op de deck: volledige poster, full screen, geen tekst.

Achtergrond: dient als totaalkaart. De navigator licht per slide de relevante posterzone op.

Slide 3. AI-agents in Fintech, via MCP

Op de deck:

- AI-agents verwerken betalingen en bedrijfskritische taken.
- Communicatie loopt via het Model Context Protocol.
- Standaardisatie opent aanvalsvlakken: prompt injection, tool poisoning, privilege escalation.
- “56% van de bedrijven met AI-agents rapporteert MCP-beveiligingsincidenten.”

Achtergrond (Inleiding, Probleemstelling; Stand van zaken, sectie 1):

- Fintech groeide de afgelopen vijf jaar met 23,58% per jaar (statista2024fintechgrowth).
- MCP is de de-facto standaard voor agent-naar-systeem-communicatie, vergelijkbaar met de rol van HTTP voor webapplicaties (anthropic2024mcpwhitepaper).
- De drie aanvalsvectoren komen uit (song2025protocolunveilingattackvectors). Confused-deputy en “Living off AI” uit (Cato2025).
- De 56%-statistiek komt uit een systematische analyse (guo2025systematicanalysismcpsecurity).
- Doelgroep: security teams, DevSecOps-engineers en enterprise architects in gereguleerde sectoren.

Slide 4. Onderzoeksvraag

Op de deck:

- Hoofdvraag: hoe kan een framework MCP-servers automatisch testen op kritieke kwetsbaarheden, en hoe verhoudt dekkingsgraad en tijd zich tot een handmatige beoordeling?
- Scope: één gecontroleerde evaluatiecase (Goat) met een vaste ground truth.

Achtergrond (Inleiding, Onderzoeksvraag en Doelstelling):

- Deelvragen probleemdomain: welke OWASP-kwetsbaarheden zijn kritiek voor Fintech, welke beperkingen hebben Burp en Semgrep, hoeveel tijd kost een handmatige

review en wat zijn de foutbronnen.

- Deelvragen oplossingsdomein: recall en precisie, het tijdsverschil, en waar menselijke expertise nodig blijft.
- Deliverables: een werkend prototype, een kwantitatieve evaluatie (recall, tijd, precisie) en evidence-based aanbevelingen met expliciete vermelding van de beperkte externe validiteit.
- De FINAL-versie spreekt bewust over één case. De “twee cases” uit de draft is geschrapt.

Slide 5. Waarom generieke tools tekortschieten

Op de deck:

- Burp Suite ziet HTTP en JSON-RPC, maar kent de semantiek van MCP-schema's niet.
- Semgrep detecteert code-patronen, maar niet de runtime-interactie.
- “Geen contextuele kennis van MCP-specifieke kwetsbaarheden.”

Achtergrond (Stand van zaken, Status quo en Bestaande tools):

- Een handmatige MCP-review kost gemiddeld 8 tot 16 uur per iteratie (Uproot2025), wat een bottleneck vormt in agile omgevingen.
- Burp onderschept JSON-RPC maar herkent geen kwaadaardige tool-beschrijving. Semgrep mist runtime-analyse.
- MCPSafetyScanner (radosevich2025mcpsafetyaudit) is LLM-agentisch, dus niet-deterministisch, en vereist een externe API (een privacyprobleem in Fintech). Jouw keuze valt op deterministische, vaste detectieregels (zie NF-2 reproduceerbaarheid).
- Invariant Labs (Invariant2025) biedt een losse scanning-tool zonder breder framework.

Slide 6. Architectuur

Op de deck: TikZ-diagram. MCP-server naar Discovery naar SAST en Fuzzing naar Rapportgenerator naar report.json en report.md, binnen een “Scanner, Docker”-container.

Achtergrond (Implementatie; Methodologie, fase 3):

- Drie onafhankelijke modules, sequentieel uitgevoerd. Ze delen één Finding-model (`rule_id`, `severity`, `tool_name`, `title`, `detail`), zodat SAST en fuzzing uniform verwerkt worden.
- Discovery roept `list_tools`, `list_prompts` en `list_resources` aan en valideert de respons via Pydantic-modellen.
- Gebouwd in Python 3.12 met de officiële `mcp`-library. Draait via Docker Compose op een geïsoleerd intern netwerk (`mcp-lab-net`).
- Twee rapporten: `report.json` (integreerbaar in CI/CD) en `report.md` (leesbaar, met badges).

Slide 7. Statisch en dynamisch

Op de deck:

- SAST (schema): prompt injection in beschrijving, ontbrekende autorisatiecontext, ongebonden parameters.
- Fuzzing (runtime): information disclosure, injection-reflectie, autorisatiebypass en tool poisoning.
- “Runtime-gedrag is onzichtbaar in broncode, daarom allebei.”

Achtergrond (Implementatie, SAST en Fuzzing; Stand van zaken, Geautomatiseerd):

- SAST telt vier regels: SAST-001 prompt injection (regex op `ignore rules, you are now`), SAST-002 ongebonden numeriek, SAST-003 ontbrekende autorisatie, SAST-004 ongebonden string.
- Fuzzing telt vier regels: FUZZ-001 information disclosure, FUZZ-002 injection-reflectie, FUZZ-003 autorisatiebypass (plus negatieve bedragen), FUZZ-004 tool poisoning (scant de respons).
- Onderbouwing van SAST, DAST en fuzzing: (`chess2004static`) en (`manes2019art`). De regels staan in losse lijsten, dus een nieuwe regel is een functie toevoegen (NF-1 modulariteit).

Slide 8. Demo (screencast)

Op de deck: still en de aanduiding demo (screencast).

Achtergrond (Validatie, Testopstelling):

- Goat bevat vijf opzettelijk ingebedde kwetsbaarheden. V1 prompt injection in de description en V2 geen autorisatie op `analyze_financial_document`. V3 geen autorisatie en V4 negatieve bedragen op `execute_transfer`. V5 tool poisoning op `fetch_market_news`.
- Volledig gecontaineriseerd op een geïsoleerd netwerk, wat een reproduceerbare ground truth geeft.

Slide 9. Resultaten op Goat

Op de deck (tabel): Recall 100% (5/5) tegenover 80% (4/5). Precisie minstens 92,3% (12/13) tegenover 100% (4/4). Tijd minder dan 30 seconden tegenover 47 minuten. Reproduceerbaarheid gegarandeerd tegenover reviewer-afhankelijk. “Het tijdsverschil van factor 90 is reviewer-onafhankelijk.”

Achtergrond (Validatie, Resultaten en Precisie):

- De scanner produceerde 13 bevindingen: SAST 7 (HIGH 3, MED 1, LOW 3) en fuzzing 6 (HIGH 6). Alle vijf de ground-truth-items werden gedetecteerd, dus recall 100%.
- Precisie is 12/13, ongeveer 92,3%. De FINAL-versie legt uit dat een klassieke FP-rate (FP gedeeld door FP plus TN) niet zinvol is in een vuln-scan, omdat true negatives niet zinvol te tellen zijn. De debateerbare vinding is FUZZ-002 op `list_recent_transactions`.
- Handmatig: 47 minuten, 4 van de 5 gevonden (V5 tool poisoning gemist), 0 false positives, dus precisie 100% (4/4).

Slide 10. Na indiening: een echte server

Op de deck: label “werk na indiening”. Zowe MCP-server (mainframe, mock-modus). Streamable HTTP, 62 tools. Ongeveer 249 bevindingen. “Precisie stort in: van 92,3% op Goat naar bijna nul hier.”

Achtergrond (niet in de thesis, sluit aan op Validatie, Beperkingen):

- Dit is werk na de indiening, bedoeld om de externe-validiteitsbeperking te toetsen die je zelf benoemt. De thesis stelt al dat de Goat-recall een bovengrens is en de benchmark deels circulair.
- Zowe is de officiële open-source mainframe-MCP-stack, gedraaid in mock-modus zonder z/OS-backend en gescand met je bestaande adapter (uitgebreid met SSE-parsing).

Slide 11. De findings volgen de policy

Op de deck (tier-tabel): read(-strict) 31/106/36. update 49/196/62. delete 55/227/74. full 62/249/83. “De bevindingen schalen met de operator-policy, niet met de beveiligingsstaat.”

Achtergrond:

- De capability-tier is het echte autorisatiemechanisme van Zowe. Hoe opener de tier, hoe meer tools, hoe meer findings, lineair. Zelfs maximaal dichtgezet blijven er 36 HIGH-meldingen over op read-only tools.
- Oorzaak: jouw heuristieken (SAST-003 en FUZZ-003) zoeken autorisatie in het tool-schema, terwijl Zowe autoriseert op sessie- en transportniveau.

Slide 12. Kritische reflectie

Op de deck:

- Circulaire benchmark: Goat is ontworpen voor de regels, dus 100% recall is best-case.
- n=1 reviewer: de handmatige vergelijking is illustratief, niet statistisch valide.
- Externe validiteit: hoge recall, lage precisie buiten de benchmark.

Achtergrond (Validatie, Beperkingen):

- Detectieregels en ground truth zijn niet onafhankelijk, dus 100% recall generaliseert niet.
- De handmatige procedure is door één reviewer in een gecontroleerde omgeving uitgevoerd, wat de statistische zeggingskracht beperkt. De thesis noemt dit bewust illustratief.
- Het reproduceerbaarheidsverschil is inherent (`chess2004static`). MCPTox (`wang2025mcptox`) wordt genoemd als methodologische referentie.

Slide 13. Aanbevelingen

Op de deck:

- Screeningslaag in CI/CD, geen vervanging van een audit.

- Dwing autorisatie af op elke gevoelige tool.
- Behandel tool-outputs als onvertrouwde invoer.
- Elke run vraagt een menselijke triage.

Achtergrond (Conclusie, Aanbevelingen):

- “Het framework vervangt geen security audit. Het verlaagt de drempel om die gericht te voeren.”
- V2 en V3 (ontbrekende autorisatie) zijn het hoogste Fintech-risico, dus authenticatie afdwingen, ongeacht of de aanroeper een mens of een agent is.
- Tool poisoning in `fetch_market_news` betekent: outputs sanitiseren zoals inputs.
- Een scan onder de 30 seconden maakt CI/CD-integratie haalbaar. Docker verlaagt de drempel.

Slide 14. Conclusie

Op de deck: haalbaar en snel, met hoge recall op de doelklasse. De grens, de precisie buiten de benchmark, is even eerlijk in kaart gebracht. Toekomstig werk: MCPTox (45 reële servers).

Achtergrond (Conclusie, Kritische reflectie en Toekomstig onderzoek):

- De hoofdvraag is beantwoord: geautomatiseerde MCP-assessment is realiseerbaar en sneller dan handmatig, bij vergelijkbare of betere recall.
- De externe validiteit is expliciet beperkt. Goat representeert niet de gemiddelde productieomgeving.
- Drie richtingen voor verder werk: meer en diverse servers (MCPTox, 45 servers en 353 tools, `wang2025mcp tox`), CI/CD-integratie, en semantische detectie van prompt injection.

Back-up-slides

- OWASP MCP Top 10 mapping (correctie op de traceability-tabel die nog LLM07, LLM01 en LLM09 gebruikt): MCP03 tool poisoning, MCP07 autorisatie, MCP02 privilege escalation, Intent Flow Subversion voor prompt injection.
- Zowe-adapter en transport.
- Scope en beperkingen (geen geauthenticeerde scanning, enkel HTTP en SSE, point-in-time).